Beyond Memorization: Training-Free Style Mixing for Variability in Handwritten Text Generation Using Writer Embedding Injection in Pretrained Diffusion Models

No Author Given

No Institute Given

Abstract. Recent advancements in handwritten text generation using diffusion models have achieved high-quality and realistic handwriting synthesis. However, existing models often suffer from limited style variability, which reduces their effectiveness for downstream tasks like Handwriting recognition, writer identification, which rely on diverse handwriting samples to ensure model generalization and robustness. Without sufficient variability, models trained on synthetic data risk overfitting to a narrow set of styles, limiting their applicability in real-world scenarios. Diffusion models typically require large-scale datasets to generalize effectively, but in the domain of handwriting generation, comparatively limited training data is available, leading to strong memorization tendencies. As a result, generated handwriting samples often replicate training data instead of introducing novel variations, further restricting their usefulness for downstream applications.

To address this, we propose a training-free style mixing approach that dynamically injects multiple writer styles at inference time, enabling controlled handwriting diversity. Our method leverages a pre-trained diffusion model, allowing flexible writer style transitions at the character level without modifying model weights. By introducing controlled style variation, our approach mitigates memorization effects and enhances the diversity of generated samples, making them more suitable for training and evaluation purposes.

Keywords: Pretrained Diffusion Models, Handwritten Text Generation, Style Mixing, Writer Embedding Injection, Training-Free Variability, Controlled Diversity

1 Introduction

Diffusion models have recently gained popularity for generating high-quality images, including handwritten text. However, despite their impressive generative capabilities, these models suffer from several critical limitations that hinder their effectiveness in handwritten text synthesis. Some of these limitations are as follows.

- **Memorization Problem & Limited Variability in Small Datasets:** Diffusion models tend to memorize patterns from the training data instead of generating diverse, novel outputs. This leads to low variability in the synthesized samples, reducing their effectiveness for real-world applications [26]. This Memorization issues are amplified when training on small datasets. Studies indicate that generalization becomes reliable only when dataset sizes exceed 2 million samples [26]. However, most handwritten text datasets like IAM [19], CVL [17] contain only 100,000–200,000 samples, making them prone to overfitting and reduced diversity in outputs.

– **Dataset Size Constraints:** Unlike large-scale generative models trained on massive datasets like LAION (400 million images) [25], handwriting datasets are significantly smaller. This dataset constraint limits the model's ability to learn diverse writing styles, resulting in repetitive, less useful outputs.

Handwritten text generation is particularly affected by these challenges due to the small dataset sizes, hence there is a need for style variability. Existing methods, such as WordStylist [20], attempt to generate synthetic handwritten word images using diffusion models. WordStylist is trained on the IAM Handwriting Dataset, which contains only 44K training samples. This dataset size is significantly smaller than large-scale generative datasets, making WordStylist susceptible to memorization issues and limited variability. Figure 1 illustrates the memorization problem in WordStylist. The odd-numbered columns (1,3,5,7) show original handwritten images, while the even-numbered columns (2,4,6,8) display synthetic images generated by WordStylist. It can be observed that the generated images closely resemble the original training samples instead of producing diverse handwriting variations. This lack of stylistic diversity makes WordStylist-generated data less useful for training robust models.

Fig. 1: Comparison of original and generated handwritten images using the WordStylist framework. Odd-numbered columns show real handwriting, while even-numbered columns display synthetic images that suffer from memorization, leading to reduced variability.

To evaluate how much the generated handwriting is affected by memorization, we use the *Handwriting Distance (HWD)* metric [23]. HWD is a perceptual distance-based evaluation score specifically designed for styled handwritten text generation. Unlike traditional metrics such as Fréchet Inception Distance (FID) [15] or Kernel Inception Distance (KID) [3], which primarily measure overall image similarity, HWD operates in a feature space trained to extract handwriting-specific style attributes. It computes the Euclidean distance between style features extracted from real and generated images, providing a fine-grained assessment of handwriting variability. This makes HWD particularly suitable for evaluating the effectiveness of style mixing techniques in handwritten text

synthesis. For diffusion-generated handwriting samples using the WordStylist [20] framework, the *HWD* score is found to be on average **0.93** across three different synthetically generated sets from IAM [19] data, indicating a strong resemblance to training data and highlighting the model's tendency to replicate rather than introduce new stylistic variations. This has serious consequences for downstream tasks like Handwritten text recognition (HTR) [24],[1],[11], where diverse training samples improve model generalization. Writer classification [14], where limited variation makes it difficult to distinguish different writers.

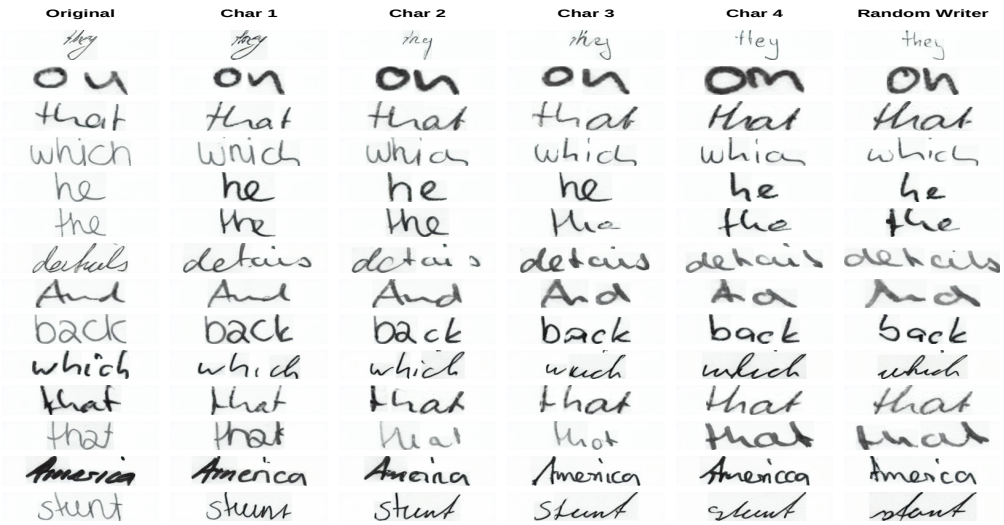

Fig. 2: Character-wise style transition in handwriting generation. The first column represents the original handwriting style, while subsequent columns progressively replace character embeddings with those from a randomly selected writer. The final column represents a fully transitioned sample generated in the style of the new writer.

Existing frameworks like WordStylist [20],DiffPen [21],One-DM[4], WordDiffusion [12] or HTLDIFF [10] lack mechanisms to introduce controlled variability. To address this issue, we propose a novel approach that injects multiple writer embeddings into the diffusion process at the character level to enhance stylistic variation and dynamically transitions between different writer styles rather than enforcing a single writer identity. Our method operates entirely at inference time, eliminating the need for retraining. Instead of conditioning the entire word on a single writer, our approach progressively mixes embeddings from two different writers, ensuring stylistic diversity while preserving coherence.

Figure 2 illustrates this character-wise style transition by text generated by our method. First column shows the original handwriting style. Subsequent columns demonstrate a gradual transition, where some characters are replaced with embeddings from a second writer. Final column Displays the fully transitioned sample in the new writer's style. The columns between the original and random writer represent images generated by our framework, showcasing the introduced variability. These intermediate transitions are the core of our proposed The images between the column "Original" and "Random Writer" are synthetically generated variable samples.

Unlike previous methods that require retraining, our approach operates entirely at inference time, making it computationally efficient. By dynamically injecting writer embeddings into the

diffusion process, we introduce controlled style mixing, allowing for diverse handwriting synthesis without modifying the base diffusion model. This enables scalable and flexible generation, crucial for handwritten text synthesis where dataset sizes are inherently small.

Our main contributions are as follows:

- **Attention-Based Character Localization:** We develop an attention-based framework that localizes character positions within the generated text. While our method applies attention across input, middle, and output layers, our experimental findings highlight that the attention maps extracted from the middle block provide the most reliable character localization. These insights guide precise writer embedding injection, ensuring controlled handwriting variations.

- **Character-Level Controlled Variability:** Unlike prior approaches that condition an entire word on a single writer embedding, we introduce character-level style mixing. By selectively injecting different writer embeddings at controlled positions, we enable diverse yet coherent handwriting variations while preserving the original word structure.

- **Enhanced HWD Evaluation:** While maintaining the original HWD metric formulation, we refine its evaluation by computing distances against both the original ground truth image and a randomly generated writer sample. This dual-reference approach ensures a more comprehensive assessment of style variation, capturing both adherence to the original writer and deviation towards a new writer style.

This structured variability allows for high-quality, diverse handwriting synthesis without requiring additional data, making our approach well-suited for real-world applications.

2 Literature Review

In recent years, substantial progress has been made in the field of synthetic text generation, particularly in word-level handwritten text synthesis. Researchers have explored a range of deep learning techniques, including adversarial networks, transformers, and diffusion models, to improve the quality and variability of generated handwriting.

A foundational study by Graves [9] pioneered the use of Recurrent Neural Networks (RNNs) for generating online handwriting trajectories. This work was later expanded upon in [31], which further refined the trajectory-based synthesis introduced by [13]. These early methods primarily focused on online handwriting generation rather than synthesizing complete handwritten word images.

Among deep generative models, Generative Adversarial Networks (GANs) have been widely applied to handwriting synthesis [28,32]. Approaches like GANwriting [16] utilize calligraphic style conditioning to integrate textual input, allowing for the generation of diverse handwritten text samples. Additionally, TS-GAN [5] incorporates pixel-level reconstruction to capture fine-grained handwriting details effectively. ScrabbleGAN [6] introduced a method for composing arbitrarily long handwritten text by assembling individual character images, although it faces limitations in replicating specific calligraphic styles. JokerGAN [30] later improved efficiency in handwriting synthesis by incorporating an awareness of text line structures.

Further advancements in GAN-based methods focused on style disentanglement and improving handwriting consistency. HiGAN [7] proposed a method that separates style and content representations, achieving improved handwriting variability through adversarial training. Expanding upon this, HiGAN+ [8] refined this technique by introducing contextual and local patch losses, which

enhance stylistic coherence and improve image quality. While these adversarial models have been effective, they often suffer from instability during training and limited generalization across unseen handwriting styles. In an effort to enhance control over handwriting generation, SLOGAN [18] was introduced as a parameterized GAN-based model capable of synthesizing text with user-specified style attributes. However, ensuring smooth stylistic transitions across longer sequences remains a challenge in such models.

Transformer-based architectures have also been explored for handwritten text generation. Models such as Handwriting Transformers (HWT) [2], VATr [22] and VATr++ [27] leverage encoder-decoder structures to capture both global handwriting styles and local character variations. These approaches provide strong control over handwriting synthesis, but they lack hierarchical representations, limiting their ability to accurately model the structure of natural handwriting.

More recently, diffusion models have gained prominence due to their ability to iteratively refine noisy inputs into realistic handwritten text. WordStylist [20] applies diffusion techniques to generate words conditioned on text and writer embeddings, though it does not explicitly model global stylistic attributes. Additionally, GlyphControl [29] employs a diffusion-based approach to synthesize text using predefined glyph templates, ensuring consistent character rendering. However, this method is primarily designed for digital text generation rather than natural handwriting synthesis.

3 Introducing Variability via Writer Embeddings

Throughout this paper, we are using the following notations:

- $\mathbf{T}$: The input text embedding specifying the content to be generated.
- $\mathbf{W}$: The writer identity embedding, which conditions the diffusion model on a specific handwriting style.
- $\mathbf{I_{GT}}$: The original handwritten image, serving as a ground truth reference for evaluating generated handwriting variability.
- $\mathbf{I}$: The image features at diffusion time step t, capturing intermediate noisy or denoised versions of the handwritten word.
- $\mathbf{I(W_o)}$: The synthetically generated image feature representation using the pretrained WordStylist diffusion framework, conditioned with the original writer embedding W_o, where the writing style corresponds to the original writer throughout the entire word.
- $\mathbf{I(W_r)}$: The synthetically generated image feature representation using the pretrained WordStylist diffusion framework, conditioned with the randomly selected writer embedding W_r, where the writing style corresponds to a different writer throughout the entire word.
- $\mathbf{I(W_o, W_r)_c}$: The synthetically generated image feature representation using the pretrained WordStylist diffusion framework, conditioned with both the original writer embedding W_o and the randomly selected writer embedding W_r, enabling style mixing up to the c-th character ($c \in \{1, 2, 3, 4\}$) with W_r, after which the original embedding is applied to the remaining characters W_o.
- $\mathbf{A}$: The set of character-level attention maps at time step t, where each map corresponds to one of the 10 possible character positions in the word. This provides spatial localization for all character positions in the generated handwriting.
- $\mathbf{A_c}$: The specific character-level attention map at time step t, focusing on the spatial localization of the c-th character in the word. This is a subset of A, extracted to highlight the region associated with character position c within the generated handwriting.

- $\mathbf{M}_c$: A dynamic masking function guided by $\mathbf{A}_c$, determining where the writer identity transitions from the original writer to a randomly selected writer.
- $\mathbf{W}_o$: The original writer identity embedding, which maintains the writer's style.
- $\mathbf{W}_r$: The randomly selected writer identity embedding.
- $\mathbf{E}_t$: The time-step embedding.

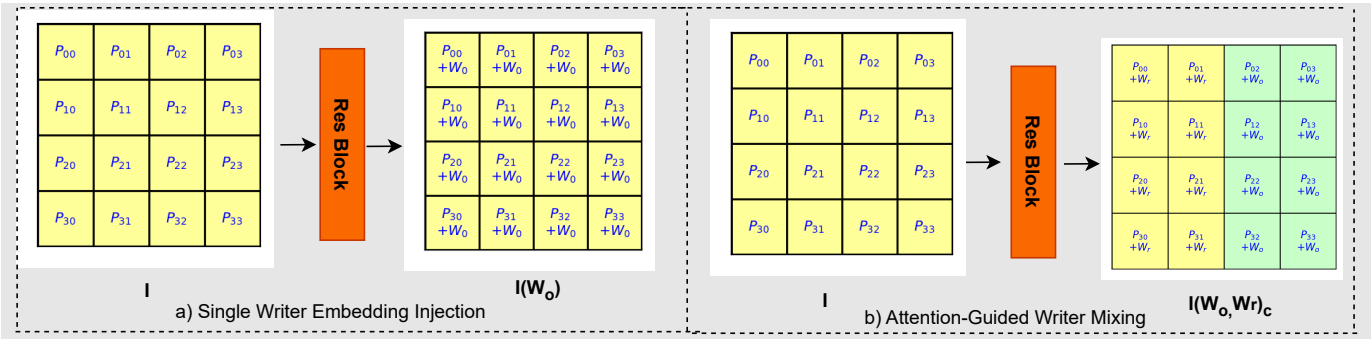

Fig. 3: Spatial Writer Embedding Injection. (a) Single Writer Embedding Injection – A single writer embedding (W_o) is injected element-wise into the image feature map (I) within the residual blocks of the U-Net, ensuring the entire handwriting follows the original writer's style at the pixel level ($P_{i,j}$). (b) Attention-Guided Writer Mixing – Attention maps are used to selectively inject different writer embeddings (W_o and W_r) at the character level, modifying the handwriting style at the pixel level ($P_{i,j}$). The transition between writer styles is controlled through attention maps, ensuring a gradual and smooth pixel-wise style change across characters.

3.1 Key Intuition for Writer Style Injection

To begin, when we use only the original writer embedding $\mathbf{W}_o$, this embedding is injected element-wise into the entire image feature map $\mathbf{I}$ within the residual blocks of the U-Net as shown in figure 3.a. The result is a new image feature map, $\mathbf{I}(\mathbf{W_o})$, ensuring that the entire generated handwriting is conditioned on the original writer's style. This injection occurs globally across all pixel locations in the image feature map, and no attention maps are required at this stage.

Spatial Control via Attention Maps: We use attention maps $\mathbf{A}_c$ (as shown in Fig. 5) to determine the approximate positions of the characters within the text. This allows us to selectively inject different writer embeddings using the random writer embedding $\mathbf{W}_r$ for some characters, while the original writer embedding $\mathbf{W}_o$ is applied to others. This ensures that variability is introduced only at specific character locations, resulting in the modified image feature map $\mathbf{I}(\mathbf{W_o}, \mathbf{W_r})_\mathbf{c}$ as shown in Figure. 3.b, which preserves the overall coherence of the handwriting while allowing for controlled style variation.

Smooth Character-Level Style Transition: By leveraging the attention maps $\mathbf{A}_c$, we assign different writer styles to different characters, enabling a gradual transition from one handwriting style to another. This transition ensures that the shift in style is smooth and coherent, even as variability is introduced character by character. The final image feature map reflects these gradual style changes, resulting in $\mathbf{I}(\mathbf{W_o}, \mathbf{W_r})_\mathbf{c}$ with a smooth blend of styles across the text. Refer fig. 3.b

for details. The images shown in Figure 6 for character positions 1, 2, and 3 represent the actual generated samples where controlled style variations have been introduced. These same images are used in the results section, where we evaluate handwriting variability across the three generated sets derived from the IAM Handwriting Dataset. By analyzing these variations, we assess the impact of writer embedding injection in generating diverse yet coherent handwriting samples.

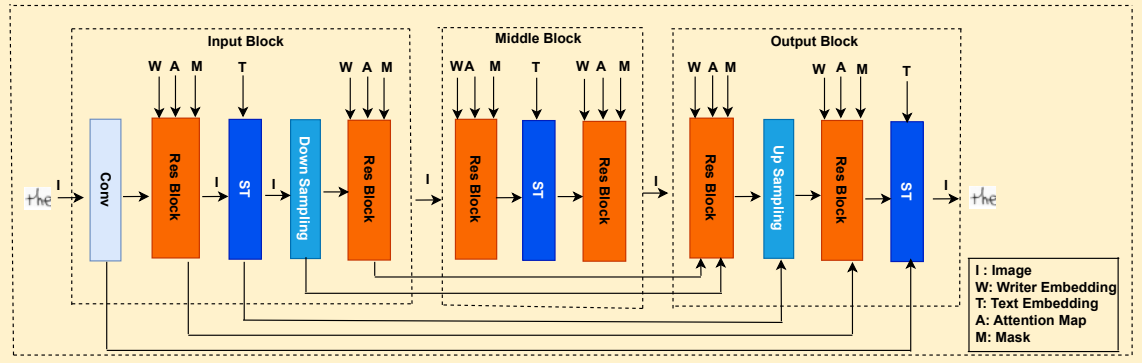

Fig. 4: U-Net architecture for handwriting generation. The model leverages attention maps and writer embeddings to introduce controlled variability. The notations used in the figure are: I (Image), W (Writer Embedding), T (Text Embedding), A (Attention Map), ST (Spatial Transformer) and M (Mask). The attention maps and writer embeddings are injected at multiple stages in the U-Net to modulate the handwriting generation process, ensuring stylistic consistency and controlled variability.

3.2 Attention Map Extraction and Character Localization

The attention map extraction process utilizes a pre-trained U-Net model as shown in Fig.4. In this model, cross-attention mechanisms are employed in Spatial Transformer (ST) block to enable interactions between the predefined textual embedding $\mathbf{T}$ and the image feature map $\mathbf{I}$. Specifically, $\mathbf{I}$ is used as queries, while $\mathbf{T}$ acts as keys and values. The attention scores computed from these interactions allow the model to align the spatial features in $\mathbf{I}$ with the textual content, forming the basis of attention maps $\mathbf{A}$.

The U-Net architecture is structured into three key components: an input layer, a middle layer, and an output layer. These layers and block inside it are shown in Fig.4, cross-attention mechanisms are applied in all these sections to integrate $\mathbf{T}$ into $\mathbf{I}$ during image generation. However, the attention maps $\mathbf{A}$ extracted from the middle block are particularly valuable because this block captures the most refined spatial and contextual representations. Unlike the input layer, which primarily focuses on low-level features, or the output layer, which integrates high-level details for final generation, the middle block strikes a balance, providing the richest representation for accurately localizing characters. This makes it ideal for extracting $\mathbf{A}$ to guide writer embedding injection and achieve stylistic variability. These $\mathbf{A}$ are tensors of shape $(\mathbf{B}, \mathbf{L}, \mathbf{H}, \mathbf{W})$, where $\mathbf{L}$ represents the maximum number of characters in a word, fixed at 10, $\mathbf{B}$ denotes the batch size, $\mathbf{H}$ is the height, and $\mathbf{W}$ is the width of the image feature map $\mathbf{I}$. For each character c, the corresponding attention map $\mathbf{A_c}$

is indexed along the character dimension, resulting in a tensor of shape $(\mathbf{B}, \mathbf{H}, \mathbf{W})$. Please refer to Fig. 5 to see character level attention maps. By analyzing these attention maps, the maximum X-coordinates $\mathbf{X}_c^{max}$ are computed for each character $\mathbf{C}$. which approximate the horizontal location of the character in the generated image. These approximate locations guide the injection of writer embeddings into $\mathbf{I}$ as shown in Figure. 3.b.

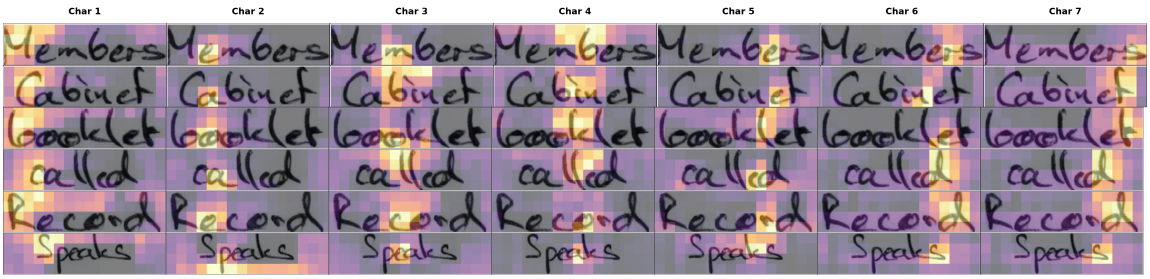

Fig. 5: Attention maps visualization for character-level localization. Each row corresponds to a different word, and each column displays the attention map for a specific character in that word. The rows represent the complete word-level attention maps $\mathbf{A}$, while each column within a row corresponds to a character-specific attention map, $\mathbf{A}_c$. These attention maps highlight the spatial focus of the model for each character, effectively guiding writer embedding injection and style transitions.

Fig. 5 illustrates this process, where each row corresponds to a word (e.g., "Members", "Cabinet", etc.), and each column displays $\mathbf{A}_c$ for a specific character in that word. These attention maps highlight the approximate regions where the model focuses for each character, demonstrating the spatial alignment between $\mathbf{I}$ and $\mathbf{T}$. Although the attention is not perfectly aligned with the characters, it localizes their approximate positions effectively, enabling precise injection of $\mathbf{W}_o$ and $\mathbf{W}_r$. To ensure consistency, when the sequence length $\mathbf{L}$ is set to 10 but the word has fewer characters, the remaining positions are padded. This ensures that all $\mathbf{A}_c$ maintain a uniform shape, simplifying the process of identifying character-specific regions in the attention maps. The horizontal coordinates extracted from these maps, denoted as $\mathbf{X}_c^{max}$, provide a dynamic reference for blending writer styles in a controlled manner. Despite its effectiveness, the localization is approximate rather than exact, requiring post-processing or smoothing to enhance precision. However, this limitation does not impede their ability to generalize to longer sequences or adapt to datasets beyond WordStylist, as the mechanism is flexible and scalable. These insights allow precise $\mathbf{W}_o$ and $\mathbf{W}_r$ injections, ensuring stylistic diversity while maintaining structural coherence in $\mathbf{I}$.

3.3 Writer Embedding Injection via Residual Blocks

In this section, we describe the mechanism for injecting writer embeddings into the residual blocks of the U-Net-based diffusion framework. This process introduces controlled variability in handwritten text generation by dynamically altering the writer identity at the character level. Unlike previous methods that condition the entire image on a single writer embedding $\mathbf{W}_o$, our approach progressively mixes a secondary writer embedding $\mathbf{W}_r$ into specific character regions.

The key steps of this process include:

Fig. 6: Attention maps and word visualization for character-level localization. Each row corresponds to a word, with odd-numbered rows displaying word images and even-numbered rows showing the corresponding attention maps. The leftmost column in odd-numbered rows represents the ground truth image (I_{GT}) from the original writer, while the rightmost column corresponds to the generated image conditioned on a randomly selected writer ($I(W_r)$). The intermediate columns show style-mixed images at specific character positions, represented as $I(W_o, W_r)_c$. Even-numbered rows visualize attention maps that highlight character-specific regions according to the column labels.

- Extracting attention maps $\mathbf{A}_c$ from the middle block of the U-Net to determine character locations See Section 3.2 for details on attention map extraction and character localization.
- Applying a masking function $\mathbf{M}_c$ to selectively inject different writer embeddings at designated character positions.
- Modifying stroke shape, letter spacing, and curvature by blending embeddings dynamically.

To summarize this process, Algorithm 1 provides a high-level pseudocode representation of writer embedding injection.

Spatial Writer Embedding Injection The writer embedding injection process occurs inside the residual blocks of the U-Net at each timestep t. Each residual block contains a transformation of the feature map $\mathbf{I}_t$, which represents the intermediate noisy or denoised version of the handwritten word. Instead of conditioning the entire image on a single writer embedding, we modify this feature representation at the spatial level by incorporating multiple embeddings based on character-wise attention localization. The attention-based localization process used for embedding injection is visualized in Figure 5, where each attention map highlights character-specific regions. These maps allow us to transition from a global writer embedding to a character-level controlled style transition.

The injection process follows these steps:

1. **Extracting Attention-Based Character Locations:** The attention maps $\mathbf{A}_c$ extracted from the middle block provide spatial information about character positions. The attention scores localize each character c along the width of the generated text, as shown in Figure 5.

Algorithm 1: Writer Embedding Injection During Diffusion Inference ($\tau = 600$ Steps)

Data: Handwritten word image I, original writer embedding W_o, random writer embedding W_r
Result: Final image with controlled style variations

Notation: - The diffusion process runs for $\tau = 600$ steps, where t represents the current diffusion step. - $I(W_o)$ represents the image with the original writer's style. - $I(W_r)$ represents the image with the random writer's style. - $I(W_o, W_r)_c$ represents the mixed image where writer embedding transition occurs at character c.

for $t = 1$ *to* τ **do**

`// Iterate over all diffusion steps`

Extract attention maps A_c from the middle block of U-Net;

for *each character c in the word* **do**

`// Iterate over each character`

Determine spatial location $X_c^{\max}$ using A_c;

Define mask $M_c(x, y)$ based on $X_c^{\max}$;

Apply embedding injection using the correct notation:

$$I(W_o, W_r)_c = (1 - M_c) \odot I(W_o) + M_c \odot I(W_r)$$

return Final generated image;

2. **Determining the Writer Embedding Transition Boundary:** Using $\mathbf{A}_c$, we compute the maximum horizontal coordinate $X_{\max}^{(c)}$ for each character c. This coordinate marks the transition boundary where the original writer embedding $\mathbf{W}_o$ transitions into the randomly selected writer embedding $\mathbf{W}_r$.
3. **Applying Character-Level Masking Function:** We define a dynamic binary mask $\mathbf{M}_c$ to control embedding injection:

$$\mathbf{M}_c(x, y) = \begin{cases} 1, & \text{if } x \leq X_{\max}^{(c)} \text{ (apply } \mathbf{W}_r) \\ 0, & \text{otherwise (apply } \mathbf{W}_o) \end{cases}$$

4. **Injecting Writer Embeddings:** The modified residual block feature map is updated with a weighted sum of embeddings:

$$\mathbf{I}(W_o, W_r)_c = (1 - \mathbf{M}_c) \odot \mathbf{I}(W_o) + \mathbf{M}_c \odot \mathbf{I}(W_r)$$

where $\mathbf{I}(W_o)$ represents the image with the original writer's style, $\mathbf{I}(W_r)$ represents the image with the random writer's style, and $\mathbf{I}(W_o, W_r)_c$ denotes the mixed image where writer embedding transition occurs at character c.

Incremental Character-Level Style Transition To ensure smooth style transitions across characters, writer embedding injection follows an incremental progression. Given a word of length L, we modify character embeddings over multiple diffusion steps:

- At step 1, the first character is assigned $\mathbf{W}_r$, while the remaining characters retain $\mathbf{W}_o$.
- At step 2, the first two characters adopt $\mathbf{W}_r$, with $L - 2$ characters retaining $\mathbf{W}_o$.
- This process iterates until 4, where the first four characters use $\mathbf{W}_r$, while the rest retain $\mathbf{W}_o$.

By incrementally modifying writer embeddings at the character level, we achieve a gradual style transition instead of abrupt stylistic shifts. The attention maps in Figure 6 illustrate this process, where the overlay highlights character-specific activation regions that guide embedding transitions.

Effect on Handwriting Variability The introduction of writer embedding variations impacts the generated handwriting in three key aspects:

1. **Stroke Style Changes:** Different writer embeddings influence stroke thickness, pen pressure, and curvature.
2. **Spacing Variations:** The injected embeddings alter letter spacing, impacting word compactness.
3. **Intra-Word Consistency:** Despite style mixing, the localized injection ensures that words remain legible and coherent.

Figure 6 visually demonstrates these effects, where different writer embeddings result in distinct stroke variations and spacing adjustments across different characters. To quantify the degree of variability introduced, we employ the HWD metric, measuring per-character differences between generated handwriting and original references.

4 Results and Analysis

In this section, we evaluate the effectiveness of the proposed writer embedding injection approach by analyzing its impact on handwriting variability. The evaluation is conducted using three generated sets, each derived from the IAM Handwriting Dataset, which contains 44K training words. For each word, we generate four character-level variations by selectively modifying the writer embeddings at character positions 1, 2, 3, and 4. We can go up to 10 characters but we have limited our experiment up to 4th character. As a result, each set expands from 44K words to 176K generated examples while preserving the overall textual structure.

Unlike previous methods that condition an entire word on a single writer embedding, our approach introduces controlled variability by selectively injecting different embeddings at the character level. This ensures stylistic diversity while maintaining text readability. Additionally, we generate three distinct sets, each created by selecting different random writers for variation. This controlled experimental setup allows us to systematically evaluate the impact of writer mixing on generated handwriting.

4.1 Word-Level HWD Computation for Style Variation Measurement

To evaluate the effectiveness of controlled style mixing, we employ the HWD metric to measure variability in generated handwriting. Instead of assuming a linear trend in variability, we analyze per-character HWD fluctuations to assess how different portions of the generated word shift towards or away from the original and random writer styles.

Per-Character HWD Computation In this section, we present a detailed evaluation of the proposed writer embedding injection approach in pretrained diffusion models. The effectiveness of this methodology is assessed using the HWD metric, which quantifies the variability introduced at

the character level. The baseline comparison is made with the standard diffusion model wordstylist [20] $\mathbf{HWD_{base}}$, which generates handwritten text without any writer-specific embedding injection.

For each character index c in the generated handwriting, we compute two separate HWD scores:

- $\mathbf{HWD}_{c,o} = d(I_{GT}, I(W_o, W_r)_c)$ — Measures the difference between the ground truth image from the IAM dataset (I_{GT}) and the generated image representation conditioned with both the original writer embedding W_o and the randomly selected writer embedding W_r at character position c where $c \in \{1, 2, 3, 4\}$.
- $\mathbf{HWD}_{c,r} = d(I(W_r), I(W_o, W_r)_c)$ — Measures the difference between the generated image conditioned on the random writer ($I(W_r)$) and the generated image representation conditioned with both the original writer embedding W_o and the randomly selected writer embedding W_r at character position c where $c \in \{1, 2, 3, 4\}$.

Figure 6 shows different handwriting variations. The leftmost column represents the ground truth image (I_{GT}) from the original writer, while the rightmost column corresponds to the generated image conditioned on a randomly selected writer ($I(W_r)$). The intermediate columns display images where writer style mixing is applied at specific character positions, represented as $I(W_o, W_r)_c$ where $c \in \{1, 2, 3\}$.

Justification of Per-Character HWD Computation Instead of using a single aggregated HWD score for an entire word, we compute per-character HWD scores for the following reasons:

- Handwriting variations are not uniform across all characters. Some characters may retain more of the original style, while others adopt features of the random writer.
- Per-character evaluation allows us to track gradual transitions in writer mixing across different sections of the word.
- Aggregating at the word level would fail to capture localized changes, making it difficult to analyze how different characters contribute to overall variability.

Per-Character Variability Computation Since the handwriting style is modified at a per-character level, we define an individual variability score for each character c by averaging its distances from both reference images:

$$\mathbf{HWD}_{c,\text{var}} = \frac{\mathbf{HWD}_{c,o} + \mathbf{HWD}_{c,r}}{2}, \quad c \in \{1, 2, 3, 4\} \tag{1}$$

This score quantifies how different the synthesized character is from both of the reference styles. Comparing with the ground truth image I_{GT} ensures that we measure deviations from the original writing style, while the comparison with the random writer image $I(W_r)$ verifies whether the model introduces variations rather than simply reproducing an alternative writer's style. Averaging both distances provides a balanced measure, ensuring that the introduced style changes are controlled and meaningful rather than arbitrary or already present in the model's learned distribution.

4.2 Evaluation of Character-Level Style Variability with and without OCR Filtering

To assess the impact of writer embedding injection on the style variability of generated handwriting, we analyze the HWD metric across different character modifications. This evaluation considers both

a general assessment (all generated samples) and a refined assessment where only OCR-correctly transcribed samples are considered.

The proposed method generates four character-wise variations per word by injecting a random writer's style into the generated text at character positions $c = \{0, 1, 2, 3\}$. The HWD metric measures the deviation of the generated handwriting from the original writer's style. Table 1 presents the computed HWD scores for different character positions across three generated sets.

Table 1: Character-Level Variability vs. Original Writer **HWD** Score. Higher values indicate greater deviation from the original writer.

Set	$\mathbf{HWD_{c=1}}$	$\mathbf{HWD_{c=2}}$	$\mathbf{HWD_{c=3}}$	$\mathbf{HWD_{c=4}}$	**Baseline** $\mathbf{HWD_{base}}$
Set 1	1.8955	1.7883	1.7232	1.6946	0.9391
Set 2	1.5241	1.5294	1.5231	1.5162	0.9391
Set 3	1.5275	1.5325	1.5266	1.5200	0.9391

The results indicate that writer embedding injection significantly enhances handwriting variability. However, measuring HWD across all generated samples does not account for OCR errors, which may introduce noise into the evaluation. To obtain a more precise estimate of handwriting variability, we conduct a refined evaluation by filtering out samples where the OCR-extracted text does not match the expected transcription. This ensures that the HWD values reflect genuine stylistic variations rather than inconsistencies in character formation. Table 2 presents the HWD results computed exclusively on OCR-passed samples. These refined results confirm that the

Table 2: Character-Level Variability for OCR-Passed Samples **HWD** Score. Higher values indicate greater deviation from the original writer.

Set	$\mathbf{HWD_{c=1}}$	$\mathbf{HWD_{c=2}}$	$\mathbf{HWD_{c=3}}$	$\mathbf{HWD_{c=4}}$	**Baseline** $\mathbf{HWD_{base}}$
Set 1 (OCR Passed)	1.4351	1.4232	1.4327	1.4367	0.95080
Set 2 (OCR Passed)	1.4315	1.4374	1.4311	1.4277	0.9508
Set 3 (OCR Passed)	1.4385	1.4404	1.4377	1.4354	0.9508

variability introduced by the writer embedding injection remains significant even when considering only OCR-correct samples. Notably, the overall HWD values are lower than those in Table 1, which suggests that some of the high-variability cases in the general evaluation were associated with OCR misinterpretations.

By comparing these evaluations, we establish that our approach effectively enhances handwriting diversity while maintaining textual correctness. This controlled variability, achieved through training-free writer embedding injection, offers a scalable solution for generating diverse handwriting samples that balance stylistic variation with readability.

4.3 Evaluation of Generated Variability using FID and KID

To quantify the effectiveness of the proposed approach in generating diverse and high-quality handwriting, we evaluate the FID and KID metrics across multiple character variations. These metrics

Table 3: FID and KID Metrics for Characters and Random Writer Variation

Metric	**FID ↓**			**KID ↓**		
I_{GT}	**Set 1**	**Set 2**	**Set 3**	**Set 1**	**Set 2**	**Set 3**
Char 1	29.73	29.3490	29.9021	0.0198	0.0194	0.0198
Char 2	29.7390	27.5412	29.8858	0.0194	0.0191	0.0194
Char 3	28.9276	28.7113	28.7518	0.0187	0.0184	0.0184
Char 4	28.4176	27.9408	28.1483	0.0182	0.0177	0.0179
wordstylist	28.8238	28.8238	28.8238	0.079	0.079	0.079
$I(W_r)$	**Set 1**	**Set 2**	**Set 3**	**Set 1**	**Set 2**	**Set 3**
Char 1	9.9682	9.7323	10.2889	0.0006	0.0005	0.0005
Char 2	10.4258	10.2388	10.1458	0.0008	0.0007	0.0007
Char 3	10.4289	10.2316	12.3262	0.0009	0.0007	0.0007
Char 4	10.3139	10.0976	10.1458	0.0008	0.0006	0.0007
wordstylist	12.7277	12.5528	12.3262	0.0008	0.0008	0.0008

provide a statistical comparison between the generated images and the original handwritten samples.

FID and KID scores are computed using feature representations extracted from an InceptionV2 model, a widely used neural network for measuring distribution similarity. The lower the FID score, the closer the generated handwriting distribution is to the original, while lower KID values indicate a better match between generated and original samples.

For each text sample, we generate four variations (Char 1 to Char 4), progressively altering the writer identity at the character level. The baseline WordStylist model is included for comparison, where images are regenerated using the original author's pre-trained model with 600 diffusion steps.

Table 3 summarizes the FID and KID scores for all variations across three different sets. Across all character variations, FID scores remain stable, indicating that the generated images maintain high fidelity to the original handwriting. Set 2 consistently produces the lowest FID and KID scores, suggesting better alignment with the original handwriting structure. The WordStylist baseline scores are included for reference, showing that the proposed approach produces similar or better variability without requiring full retraining. The progressive injection of a random writer's identity impacts FID and KID, demonstrating controlled variability while maintaining stylistic coherence.

These results confirm that the distribution of the generated handwriting remains comparable to the WordStylist baseline while introducing controlled variability at the character level. The HWD metric, presented in earlier sections, has already validated the introduction of stylistic variation into the generated samples. By maintaining similar FID and KID scores to the baseline while achieving proven variability through HWD, our approach ensures that the generated handwriting remains realistic and diverse without altering the model's statistical distribution.

These findings validate that our approach introduces controlled diversity into generated handwriting samples without compromising realism. By allowing dynamic character-wise style blending, we produce diverse, high-quality handwriting datasets suitable for downstream tasks such as handwriting recognition and writer classification.

Dataset	CER (%)	WER (%)
All Data	4.64	13.21
Set 1	4.73	13.45
Set 2	4.9	14.13
Set 3	4.877	13.6
Baseline	4.86	14.11

Table 4: Comparison of CER and WER across different sets.

4.4 Enhancing OCR Performance by Integrating Synthetic Handwriting Variability into Base Data

The results in Table 4 demonstrate the impact of incorporating synthetically generated handwriting data into the training set of HTR. We have used HTR [24] for recognition. The baseline represents training with only the IAM dataset, while the other entries show the effect of adding synthetic variability. Notably, when synthetic data is included, both CER and WER decrease, indicating that the model benefits from increased handwriting diversity. This validates the effectiveness of the proposed method in enhancing generalization. The observed improvements highlight the role of synthetic data in bridging the domain gap and improving recognition robustness.

5 Discussion and Conclusion

The proposed approach introduces character-level style variability in handwritten text generation while preserving content integrity. The quantitative results demonstrate that controlled writer embedding injection significantly increases handwriting diversity, outperforming traditional diffusion-based handwriting generation. This enhanced variability has direct implications for HTR models, where diverse handwriting styles are crucial for improving generalization and robustness. The inclusion of synthetic handwriting with controlled style variations has been shown to reduce CER and WER, indicating that such data augmentation strategies can bridge domain gaps and enhance model performance in real-world scenarios. Future work may explore adaptive writer selection mechanisms to optimize style transitions further and investigate the integration of synthetic handwriting data into large-scale HTR training pipelines to assess long-term improvements in recognition accuracy.

6 Conclusion

In this work, we introduced a training-free writer embedding injection approach that enables character-level handwriting variability in diffusion-based handwritten text generation. Our framework effectively leverages attention-based character localization, identifying that the middle block attention maps provide the most reliable spatial information for controlled writer style mixing. Additionally, we enhanced the HWD metric by computing distances against both the original ground truth and a randomly generated writer sample, ensuring a comprehensive evaluation of style variability. The benefits of this approach extend to HTR applications, where controlled style diversity can aid in mitigating biases towards specific handwriting patterns, thus improving recognition accuracy across different writers. Experimental results confirm that our method significantly enhances handwriting diversity while maintaining coherence, offering a scalable solution for generating high-quality synthetic handwriting without additional training data.

References

1. Bhatt, R., Rai, A., Chanda, S., Krishnan, N.C.: Pho(SC)-CTC—a hybrid approach towards zero-shot word image recognition. International Journal on Document Analysis and Recognition (IJDAR) (Jul 2022)
2. Bhunia, A.K., Khan, S.H., Cholakkal, H., Anwer, R.M., Khan, F.S., Shah, M.: Handwriting transformers. 2021 IEEE/CVF International Conference on Computer Vision (ICCV) pp. 1066–1074 (2021), https://api.semanticscholar.org/CorpusID:233181822
3. Bińkowski, M., Sutherland, D.J., Arbel, M., Gretton, A.: Demystifying mmd gans. In: International Conference on Learning Representations (ICLR) (2018)
4. Dai, G., Zhang, Y., Ke, Q., Guo, Q., Huang, S.: One-dm: One-shot diffusion mimicker for handwritten text generation. In: Leonardis, A., Ricci, E., Roth, S., Russakovsky, O., Sattler, T., Varol, G. (eds.) Computer Vision – ECCV 2024, Lecture Notes in Computer Science. vol. 15116. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-73636-0_24, https://doi.org/10.1007/978-3-031-73636-0_24
5. Davis, B.L., Tensmeyer, C., Price, B.L., Wigington, C., Morse, B., Jain, R.: Text and style conditioned gan for the generation of offline-handwriting lines. ArXiv **abs/2009.00678** (2020), https://api.semanticscholar.org/CorpusID:221448289
6. Fogel, S., Averbuch-Elor, H., Cohen, S., Mazor, S., Litman, R.: Scrabblegan: Semi-supervised varying length handwritten text generation. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 4323–4332 (2020), https://api.semanticscholar.org/CorpusID:214623091
7. Gan, J., Wang, W.: Higan: Handwriting imitation conditioned on arbitrary-length texts and disentangled styles. In: AAAI Conference on Artificial Intelligence (2021), https://api.semanticscholar.org/CorpusID:235306524
8. Gan, J., Wang, W., Leng, J., Gao, X.: Higan+: Handwriting imitation gan with disentangled representations. ACM Trans. Graph. **42**(1) (2022). https://doi.org/10.1145/3550070, https://doi.org/10.1145/3550070
9. Graves, A.: Generating sequences with recurrent neural networks. ArXiv **abs/1308.0850** (2013), https://api.semanticscholar.org/CorpusID:1697424
10. Gurav, A., Chanda, S., Krishnan, N.C.: Htldiff: Handwritten text line generation - a diffusion model perspective. 2024 IEEE Thirteenth International Conference on Image Processing Theory, Tools and Applications (IPTA) pp. 1–6 (2024), https://api.semanticscholar.org/CorpusID:274180352
11. Gurav, A., Jensen, J., Krishnan, N.C., Chanda, S.: Respho(sc)net: A zero-shot learning framework for norwegian handwritten word image recognition. In: Pertusa, A., Gallego, A.J., Sánchez, J.A., Domingues, I. (eds.) Pattern Recognition and Image Analysis. pp. 182–196. Springer Nature Switzerland, Cham (2023)
12. Gurav, A., Krishnan, N.C., Chanda, S.: Word-diffusion: Diffusion-based handwritten text word image generation. In: Antonacopoulos, A., Chaudhuri, S., Chellappa, R., Liu, C.L., Bhattacharya, S., Pal, U. (eds.) Pattern Recognition. pp. 53–72. Springer Nature Switzerland, Cham (2025)
13. Ha, D.R., Eck, D.: A neural representation of sketch drawings. ArXiv **abs/1704.03477** (2017), https://api.semanticscholar.org/CorpusID:8968704
14. He, S., Schomaker, L.: Gr-rnn: Global-context residual recurrent neural networks for writer identification. Pattern Recognit. **117**, 107975 (2021), https://api.semanticscholar.org/CorpusID:233210164
15. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: Gans trained by a two time-scale update rule converge to a local nash equilibrium. In: Advances in Neural Information Processing Systems (NeurIPS) (2017)
16. Kang, L., Riba, P., Wang, Y., Rusiñol, M., Forn'es, A., Villegas, M.: Ganwriting: Content-conditioned generation of styled handwritten word images. ArXiv **abs/2003.02567** (2020), https://api.semanticscholar.org/CorpusID:212414769
17. Kleber, F., Fiel, S., Diem, M., Sablatnig, R.: Cvl-database: An off-line database for writer retrieval, writer identification and word spotting. In: 2013 12th International Conference on Document Analysis and Recognition. pp. 560–564 (2013). https://doi.org/10.1109/ICDAR.2013.117

18. Luo, C., Zhu, Y., Jin, L., Li, Z., Peng, D.: Slogan: Handwriting style synthesis for arbitrary-length and out-of-vocabulary text. IEEE Transactions on Neural Networks and Learning Systems **34**, 8503–8515 (2022), https://api.semanticscholar.org/CorpusID:247058493
19. Marti, U.V., Bunke, H.: The iam-database: an english sentence database for offline handwriting recognition. International Journal on Document Analysis and Recognition **5**, 39–46 (2002), https://api.semanticscholar.org/CorpusID:29622813
20. Nikolaidou, K., Retsinas, G., Christlein, V., Seuret, M., Sfikas, G., Smith, E.B., Mokayed, H., Liwicki, M.: Wordstylist: Styled verbatim handwritten text generation with latent diffusion models. In: Fink, G.A., Jain, R., Kise, K., Zanibbi, R. (eds.) Document Analysis and Recognition - ICDAR 2023. pp. 384–401. Springer Nature Switzerland, Cham (2023)
21. Nikolaidou, K., Retsinas, G., Sfikas, G., Liwicki, M.: Diffusionpen: Towards controlling the style of handwritten text generation. In: Leonardis, A., Ricci, E., Roth, S., Russakovsky, O., Sattler, T., Varol, G. (eds.) Computer Vision – ECCV 2024. pp. 417–434. Springer Nature Switzerland, Cham (2025)
22. Pippi, V., Cascianelli, S., Cucchiara, R.: Handwritten text generation from visual archetypes. 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 22458–22467 (2023), https://api.semanticscholar.org/CorpusID:257766680
23. Pippi, V., Quattrini, F., Cascianelli, S., Cucchiara, R.: Hwd: A novel evaluation score for styled handwritten text generation. ArXiv **abs/2310.20316** (2023), https://api.semanticscholar.org/CorpusID:264806135
24. Retsinas, G., Sfikas, G., Gatos, B., Nikou, C.: Best practices for a handwritten text recognition system. In: International Workshop on Document Analysis Systems (2022), https://api.semanticscholar.org/CorpusID:248949489
25. Schuhmann, C., Vencu, R., Beaumont, R., Kaczmarczyk, R., Mullis, C., Katta, A., Coombes, T., Jitsev, J., Komatsuzaki, A.: Laion-400m: Open dataset of clip-filtered 400 million image-text pairs. arXiv preprint arXiv:2111.02114 (2021)
26. Somepalli, G., Singla, V., Goldblum, M., Geiping, J., Goldstein, T.: Understanding and mitigating copying in diffusion models. Advances in Neural Information Processing Systems **36**, 47783–47803 (2023)
27. Vanherle, B., Pippi, V., Cascianelli, S., Michiels, N., Van Reeth, F., Cucchiara, R.: Vatr++: Choose your words wisely for handwritten text generation. IEEE Transactions on Pattern Analysis and Machine Intelligence (2024)
28. Vögtlin, L., Drazyk, M., Pondenkandath, V., Alberti, M., Ingold, R.: Generating synthetic handwritten historical documents with ocr constrained gans. ArXiv **abs/2103.08236** (2021), https://api.semanticscholar.org/CorpusID:232233484
29. Yang, Y., Gui, D., Yuan, Y., Ding, H., Hu, H.R., Chen, K.: Glyphcontrol: Glyph conditional control for visual text generation. ArXiv **abs/2305.18259** (2023), https://api.semanticscholar.org/CorpusID:258960586
30. Zdenek, J., Nakayama, H.: Jokergan: Memory-efficient model for handwritten text generation with text line awareness. Proceedings of the 29th ACM International Conference on Multimedia (2021), https://api.semanticscholar.org/CorpusID:239011850
31. Zhang, X.Y., Yin, F., Zhang, Y., Liu, C.L., Bengio, Y.: Drawing and recognizing chinese characters with recurrent neural network. IEEE Transactions on Pattern Analysis and Machine Intelligence **40**, 849–862 (2016), https://api.semanticscholar.org/CorpusID:3708198
32. Zhu, J.Y., Park, T., Isola, P., Efros, A.A.: Unpaired image-to-image translation using cycle-consistent adversarial networks. 2017 IEEE International Conference on Computer Vision (ICCV) pp. 2242–2251 (2017), https://api.semanticscholar.org/CorpusID:206770979